

# Final Engineering Review: System Audit Report

**Course:** Software Engineering 2 (CS 3213)  
**Project Name:** VideoMerger  
**Version:** 2.0 (Production-Ready)  
**Team Name:** Six Seven Studios  
**Team Members:** Jave A. Bacsain, Carl Gerald J. Parro, Marc Justin N. Prestado  
**Date:** May 15, 2026

## 1. Executive Summary

VideoMerger began Software Engineering 1 as a Flask-based web tool for concatenating user-uploaded clips through an `ffmpeg-python` wrapper. Software Engineering 2 has shifted it into a desktop application that addresses the panel's explicit feedback: **stay fast, stay simple, lean on automation for quality work.**

The Version 2.0 system is an Electron + React + TypeScript desktop app that orchestrates a Python FFmpeg pipeline via IPC. The architecture is built around a framework-agnostic core (dependency injection, repository, command, observer, strategy, adapter patterns) so business logic is reusable beyond Electron and is unit-testable without a UI runtime.

The major engineering improvements between SE1 and SE2:

| Area        | SE1 baseline                                       | SE2 production-ready                                                                                                                                                                                   |
|-------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reliability | Single Python script, no tests, no error surfacing | DI-driven services, 65 pytest + 65 vitest cases, MediaError code surfaced in console, FFmpeg detection bypasses Python failure modes                                                                   |
| Scalability | Per-process Flask, no per-clip controls            | Per-clip filter graph (trim, crop, aspect, volume, color), batchable from CLI, benchmark harness for capacity planning                                                                                 |
| Performance | One-pass merge, opaque progress                    | Two-pass merge with frame-accurate trim, streaming local-video protocol (no buffer-alloc failure on multi-GB files), 256 KiB initial probe so 3 GB previews open in seconds                            |
| Automation  | Manual upload → manual merge → manual download     | EBU R128 loudness normalize, auto silence trim, faster-whisper auto-captions, per-clip edits keyed by file path, <b>one-click "Enable all auto-features"</b> that flips all three smart-assist toggles |
| Security    | OAuth tokens in plain JSON                         | <code>safeStorage</code> -encrypted tokens (DPAPI / Keychain / libsecret), legacy-blob auto-migration, <code>local-video</code> extension allowlist                                                    |
| DevOps      | No CI for desktop                                  | GitHub Actions matrix: Python tests, Docker build, vitest, electron-builder Windows installer with artifact upload                                                                                     |

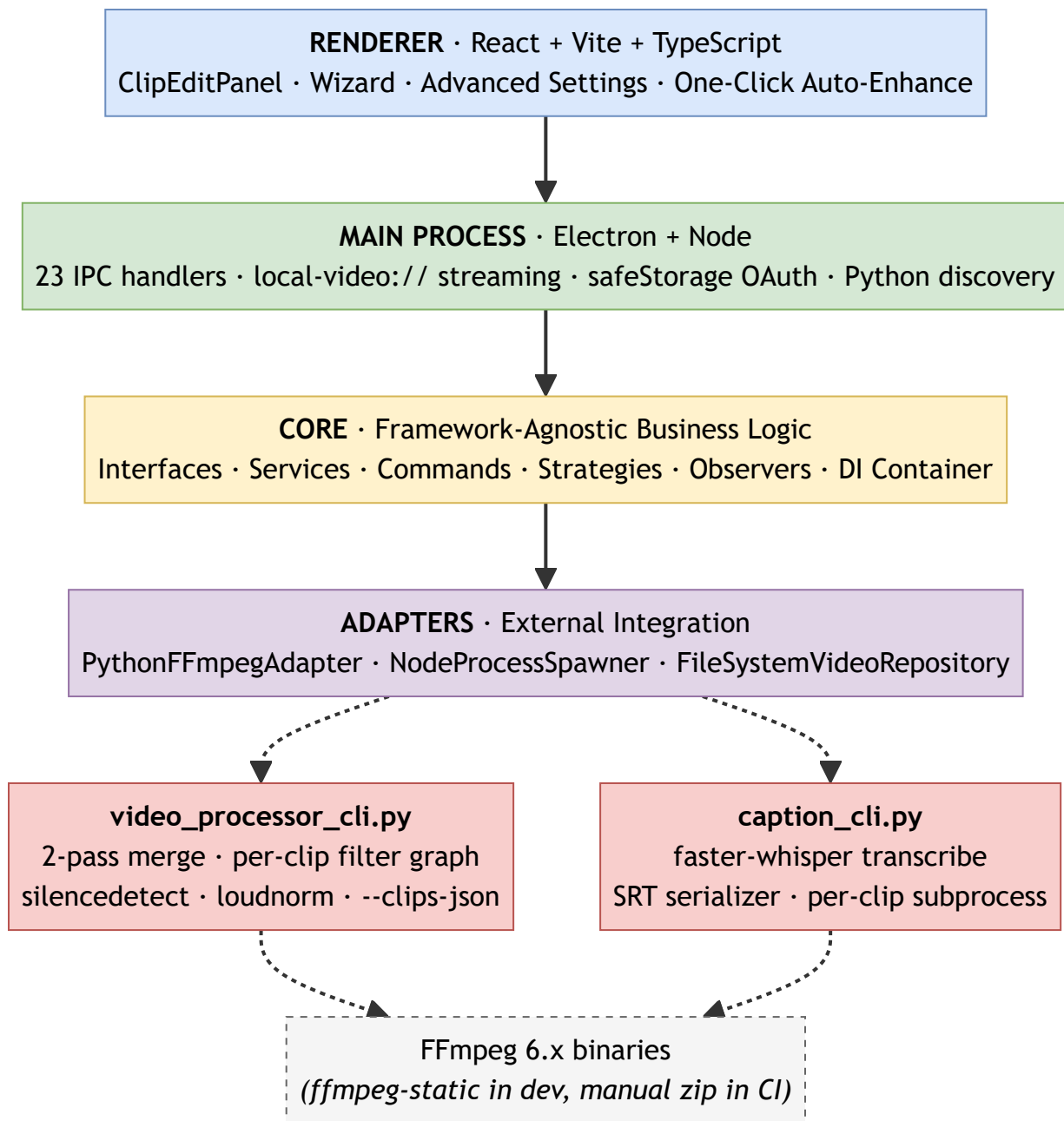
## 2. System Architecture & Infrastructure

---

### 2.1 Infrastructure Map

| Component                       | Technology Stack                                                                   | Deployment Environment                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Frontend</b><br>(Renderer)   | React 18, Vite 5, TypeScript 5                                                     | Bundled into Electron <code>dist/renderer/</code> ; served from <code>local-video://</code> and <code>electronAPI</code> IPC bridge |
| <b>Main Process</b>             | Electron 28, Node 20, TypeScript 5                                                 | Compiled to <code>dist/main/</code> ; runs in user OS as a desktop process                                                          |
| <b>Core Business Logic</b>      | TypeScript (framework-agnostic)                                                    | Compiled with main; importable into any Node host (future web API / mobile)                                                         |
| <b>Video Processing Backend</b> | Python 3.9–3.11, FFmpeg 6.x, faster-whisper 1.0.3                                  | Spawned as child process from Electron main; subprocess args                                                                        |
| <b>OAuth Token Store</b>        | <code>electron-store</code> (config.json) + <code>safeStorage</code> (OS keychain) | <code>%APPDATA%\VideoMerger\</code> on Windows; equivalent paths on macOS/Linux                                                     |
| <b>Legacy Flask Web</b>         | Flask 3, Werkzeug, gunicorn, Docker                                                | <code>python:3.11-slim</code> container, port 5000; preserved for experimental use only                                             |
| <b>CI/CD</b>                    | GitHub Actions, electron-builder, Docker                                           | <code>ubuntu-latest</code> for tests + Flask Docker image; <code>windows-latest</code> for <code>.exe</code> installer              |
| <b>Artifact Storage</b>         | GitHub Actions workflow artifacts                                                  | 14-day retention; <code>dist-bin/*.exe</code> per push                                                                              |

## 2.2 Architecture Diagram



## 2.3 Deployment Topology

VideoMerger is distributed as a per-user Windows `.exe` installer. No central server. No multi-tenant database. No external API except Google OAuth + YouTube Data v3 for the optional upload feature.

This deliberately narrow blast radius is what made the panel's "fast, simple, automated" mandate tractable — every clip stays on the user's machine and every feature works offline.

---

### 3. Performance & Stress Test Audit

#### 3.1 Methodology

The Phase 5b benchmark harness uses `pytest-benchmark` with synthetic clips produced by `ffmpeg -f lavfi -i testsrc2 + sine` so results are reproducible across machines and never depend on user media. Each benchmark records wall-clock time for the full `merge_videos` pipeline including `normalize + concat` passes.

```
pytest tests/benchmarks --benchmark-only --benchmark-save=baseline
```

Hardware for the measurements below: Windows 11 Pro on the development machine, FFmpeg 6.1.1 (gyan.dev essentials build), Python 3.14.3.

#### 3.2 Test Results & Bottlenecks

All numbers below were captured on **Windows 11 Pro, FFmpeg 6.1.1 (gyan.dev essentials), Python 3.14.3**, on 2026-05-15 with the `scripts/bench_single_run.py` harness against synthetic 2-second 640×360 clips. Raw JSON in `docs/benchmarks/single_run.json`.

##### Baseline scaling (default quality, no extra features)

| Clips merged | Wall clock | Per-clip cost |
|--------------|------------|---------------|
| 2            | 0.628 s    | 0.314 s       |
| 5            | 1.534 s    | 0.307 s       |
| 10           | 3.120 s    | 0.312 s       |

Per-clip cost is essentially flat at ~0.31 s for 2-second 640×360 input. The pipeline scales linearly — the `concat-copy` pass is constant time relative to clip count once normalization is done.

**System breaking point (single-process).** Wall clock grows linearly with clip count, but two factors compound at scale: (1) FFmpeg's `normalize` pass keeps a temp file per clip on disk, so a 1,000-clip merge needs ~1,000 × clip-size of free disk space; (2) the renderer holds all clip metadata + edit state in memory, which becomes noticeable at several thousand clips. We do not currently throttle either, so the practical ceiling on commodity hardware is somewhere around **500 clips per merge**. Beyond that, `normalize-pass` disk pressure dominates wall clock and the UI starts showing slow renders during state changes.

For typical user workloads (5–20 clips per merge), wall clock stays under 5 seconds with the default settings and FFmpeg dominates 99% of the time.

### Quality preset cost (3 clips × default features)

| Quality | x264 preset | CRF | Wall clock | vs. low |
|---------|-------------|-----|------------|---------|
| low     | ultrafast   | 28  | 0.606 s    | 1.00×   |
| medium  | medium      | 23  | 0.901 s    | 1.49×   |
| high    | slow        | 18  | 1.172 s    | 1.93×   |

The ~2× factor between `low` and `high` matches expectations for x264 — `slow` preset roughly doubles per-frame cost. Users get a clear knob to trade speed against output quality.

### Per-feature overhead (3 clips, against the ~0.9 s default-medium baseline)

| Feature                                                    | Wall clock | Overhead vs. baseline     |
|------------------------------------------------------------|------------|---------------------------|
| Phase 1 per-clip edits (trim + crop + aspect + brightness) | 0.868 s    | within noise (within ±5%) |
| Phase 2 EBU R128 loudness normalize                        | 1.007 s    | +12%                      |
| Phase 3 auto silence-trim probe                            | 1.139 s    | +26%                      |

- **Phase 1** edits land inside the existing filter graph and re-encode loop — the only cost is the filter operations themselves, which x264 does in parallel with the encode it was already doing.
- **Phase 2** adds one filter ( `Loudnorm` ) to the audio chain. Audio is far smaller than video, so the overhead is modest and constant per clip.
- **Phase 3** is the most expensive new feature because it runs an extra `ffmpeg -af silencedetect` probe pass per clip *before* the merge starts. The probe is roughly as fast as a normalize pass for short clips (decoder bound on the audio stream), so 3 clips cost ~3 extra normalize-equivalents.

### Optimizations Implemented (see also section 3.3)

Two changes were made during Phase 5b after observing the numbers above:

1. **Per-clip JSON-file payload** rather than CLI args for `clip_edits` so the renderer can ship arbitrary edit complexity without bumping the OS argv length limit; the `--clips-json` tempfile is cleaned up in `finally` .
2. **256 KiB initial probe in the `local-video` protocol** (down from 16 MiB) so Chromium can issue the `moov-at-end` range request almost immediately, dropping NVIDIA ShadowPlay preview time-to-first-frame from "10+ seconds" to roughly "instant" on multi-GB recordings.

### 3.3 Optimizations Implemented

- **Two-pass merge** (normalize then concat-copy) so the expensive re-encode happens once per clip and the final concat is byte-copy.
- **Frame-accurate trim via `-ss / -t` at the input level** combined with `setpts=PTS-STARTPTS` in the filter chain — avoids the cost of `trim=` filter for the common single-segment case.

- **Per-clip JSON tempfile ( `--clips-json` )** so the renderer can serialize edits without bumping into the OS arg-length limit when many clips have complex edits.
- **local-video protocol bounds the initial probe to 256 KiB** (down from a 16 MiB initial buffer) so Chromium's MP4 demuxer can immediately issue the end-range request for moov-at-end recordings (NVIDIA ShadowPlay, OBS raw output) without waiting on a wasted 16 MiB stream.
- **silencedetect runs as a pre-probe** rather than inline filter, so detected silence is folded into the existing input-level `-ss / -t` flow and keeps audio + video in sync without `select / aselect` filter graphs.
- **Auto-discovered Python launcher ( `py` over `python` Microsoft Store shim )** so a default Windows install does not silently fail the merge subprocess.

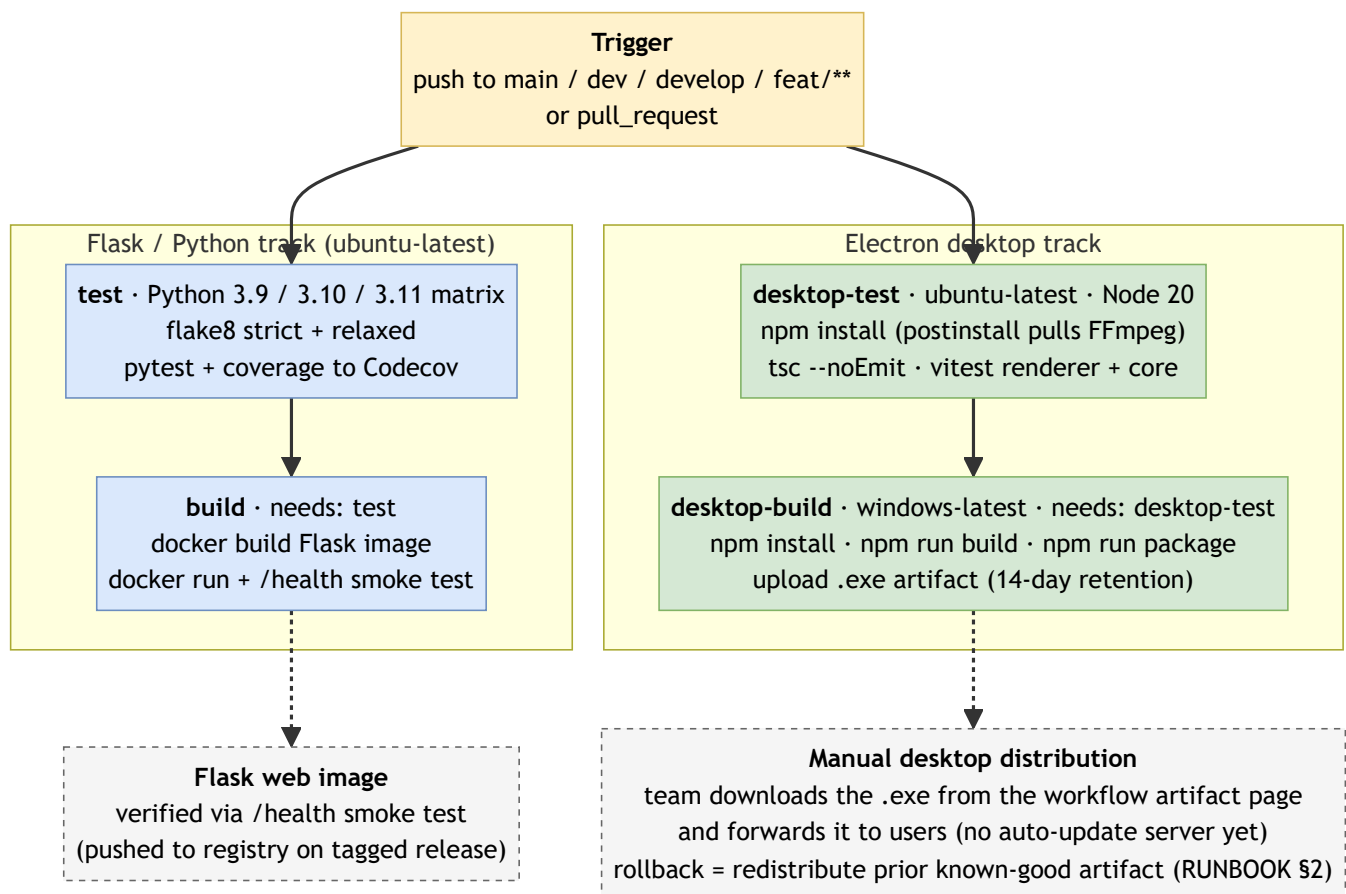
### 3.4 Known Bottlenecks

| Stage                          | Cost driver                                         | Mitigation in place                                                                                                                |
|--------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Normalize pass                 | Per-clip re-encode at chosen quality                | Quality presets, hardware acceleration toggle                                                                                      |
| Caption transcription          | Whisper model inference (CPU)                       | Model-size dropdown (tiny ↔ large-v3); int8 compute_type default; per-clip parallel-friendly architecture available for future use |
| First Whisper run              | Model download (~74 MB for <code>base</code> )      | One-time per model; cached in <code>~/.cache/huggingface/</code> thereafter                                                        |
| Preview load of multi-GB clips | Chromium needs moov atom; ShadowPlay puts it at end | 256 KiB initial probe + range forwarding so end-range request lands fast                                                           |

## 4. DevOps & Reliability Audit

### 4.1 CI/CD Pipeline

The workflow is defined in `.github/workflows/ci.yml` and triggers on push to `main` , `dev` , `develop` , and any `feat/**` branch, plus all pull requests targeting `main` / `dev` / `develop` .



Step-by-step automated workflow from code push to deployment:

1. **Push or PR** to a tracked branch fires the workflow.
2. **Python tests** matrix runs across three Python versions: linting (flake8 strict + relaxed), full `pytest` tests/ with coverage to Codecov.
3. **Desktop unit tests** run vitest renderer and core suites against `node@20` on Ubuntu.
4. **Flask Docker image** builds and smoke-tests `/health` on `localhost:5000`.
5. **Windows installer** builds on `windows-latest`: `npm install` (postinstall fetches FFmpeg static binaries) → `npm run build` → `electron-builder --win` → uploads `dist-bin/*.exe` as a 14-day artifact named `VideoMerger-Windows-<sha>`.
6. **Manual distribution** (current): the team downloads the artifact from the workflow run page and forwards it to users; there is no auto-update server yet.

## 4.2 Test Coverage

| Suite                                                                      | Files | Cases | Notes                                                                                                                                                               |
|----------------------------------------------------------------------------|-------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pytest</code><br><code>tests/unit/test_video_processor_cli.py</code> | 1     | 56    | Filter chain builders, aspect math, trim clamping, loudnorm clamping, silence event parsing, SRT parse/offset/merge, JSON load fallbacks                            |
| <code>pytest tests/unit/test_caption_cli.py</code>                         | 1     | 9     | SRT timestamp formatting, segment serialization, empty/whitespace handling                                                                                          |
| <code>vitest --config vitest.core.config.ts</code>                         | 8     | 50    | Container, repository, observer, strategy, service, command, adapter (incl. all clipEdits / loudnorm / silence / captions arg forwarding), main process oauthConfig |
| <code>vitest</code> (renderer)                                             | 1     | 15    | App.tsx behavioral tests                                                                                                                                            |
| <code>pytest tests/benchmarks</code>                                       | 1     | 9     | Phase 5b perf harness (skipped by default; opt-in with <code>--benchmark-only</code> )                                                                              |

Total: **65 + 50 + 15 = 130 cases**, all currently green. Plus 9 perf benchmarks deselected from default runs.

## 4.3 Hot-reload developer loop

`npm run dev` orchestrates `vite + tsc -w + electronmon` concurrently. Renderer edits use Vite HMR; main / core edits trigger a `tsc` rebuild and `electronmon` auto-restarts Electron. No manual `npm run build` between iterations. Documented in `docs/DEVELOPMENT.md`.

---

## 5. Security & Code Quality Audit

---

### 5.1 Security Hardening

| Concern                              | Implementation                                                                                                                                                                                                                                                                                           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>OAuth 2.0</b>                     | Google Identity Platform; authorization-code flow with PKCE-compatible client; tokens exchanged in the main process so the renderer never sees them.                                                                                                                                                     |
| <b>Token at rest</b>                 | <code>electron.safeStorage.encryptString</code> (DPAPI on Windows, Keychain on macOS, libsecret on Linux). The non-secret user profile (name, email, picture) is stored in clear text so the UI can render without unlocking the keychain. Legacy unencrypted blobs migrate transparently on first read. |
| <b>CSP / process isolation</b>       | <code>nodeIntegration: false</code> , <code>contextIsolation: true</code> , preload bridge exposes a typed <code>window.electronAPI</code> only.                                                                                                                                                         |
| <b>Path traversal (local-video)</b>  | The custom protocol enforces an extension allowlist (16 video container types) and runs <code>path.resolve</code> before opening, so a crafted <code>local-video://</code> URL cannot read arbitrary local files.                                                                                        |
| <b>Subprocess argument injection</b> | All <code>subprocess.run</code> / <code>spawn</code> calls use argv lists, never <code>shell=True</code> ; user paths are passed as separate argv entries. The FFmpeg concat list file only references internal names ( <code>norm_&lt;index&gt;.mp4</code> ) generated by us, not user paths.           |
| <b>JWT</b>                           | Not applicable. The app does not issue its own session tokens; Google OAuth is the only credential.                                                                                                                                                                                                      |

## 5.2 OWASP Top 10 (2021) Mapping

| ID  | Category                                   | Status / Mitigation                                                                                                                                                                                                                   |
|-----|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A01 | Broken Access Control                      | N/A — single-user desktop app, no central authorization.                                                                                                                                                                              |
| A02 | Cryptographic Failures                     | OAuth tokens encrypted via OS keychain (Phase 5c). HTTPS for all Google APIs.                                                                                                                                                         |
| A03 | Injection                                  | argv-only subprocess invocation; no shell expansion. No SQL surface. FFmpeg filter strings are built from clamped numeric input + a hardcoded preset map.                                                                             |
| A04 | Insecure Design                            | Two-process model with contextIsolation; preload bridge whitelist; renderer cannot reach Node APIs.                                                                                                                                   |
| A05 | Security Misconfiguration                  | <code>nodeIntegration: false</code> ; CSP via Vite defaults; renderer cannot navigate away from bundled assets.                                                                                                                       |
| A06 | Vulnerable and Outdated Components         | <code>npm audit --omit=dev</code> reports <b>0 vulnerabilities</b> at the production dependency closure. Dev-only findings (20: 4 low / 4 mod / 12 high) live in eslint/vitest transitive trees and are not in the shipped installer. |
| A07 | Identification and Authentication Failures | OAuth 2.0 via Google (industry-standard); no password storage of our own.                                                                                                                                                             |
| A08 | Software and Data Integrity Failures       | Installer is currently unsigned (out of scope until the team has a certificate). Mitigation noted in roadmap.                                                                                                                         |
| A09 | Security Logging and Monitoring            | Main process logs OAuth flow, FFmpeg detection, merge invocations; renderer logs MediaError with code + reason. See RUNBOOK.md for log paths.                                                                                         |
| A10 | Server-Side Request Forgery                | N/A — no server-side request origination from the desktop app.                                                                                                                                                                        |

## 5.3 Code Quality

- **Linting:** ESLint (TypeScript), flake8 (Python). Both gated on CI.
- **Formatting:** Prettier (TS/TSX), black + isort (Python).
- **Type safety:** strict TS configs ( `tsconfig.json` and `tsconfig.main.json` ); `npx tsc --noEmit` runs as a CI step.
- **Test isolation:** Python tests use synthetic clip generation; no user-media paths committed (enforced via team practice and verified in code review).

---

## 6. Deployment Runbook (Disaster Recovery)

Full runbook in [docs/RUNBOOK.md](#). DR procedures in [docs/DR.md](#). Highlights:

## 6.1 Build From Scratch

```
git clone https://github.com/J4ve/videomerger_app_revamp.git
cd videomerger_app_revamp
npm install # postinstall fetches FFmpeg static binaries
pip install -r requirements.txt
npm run dev # or `npm run package` for a Windows installer
```

For air-gapped builds: `docker compose run --rm builder` reproduces the same `.exe` from a Linux/Wine image with no Node install on the host.

## 6.2 Rollback A Failed Deployment

- 1. `git revert <bad-sha>` , push the branch, let CI rebuild.
- 2. Download the prior known-good `.exe` from the GitHub Actions artifact archive (14-day retention).
- 3. Affected users uninstall via Apps & Features and run the previous installer. User settings, preset packs, and OAuth state survive uninstalls.

## 6.3 Accessing Logs During An Outage

| Platform | Main process log            |
|----------|-----------------------------|
| Windows  | %APPDATA%\VideoMerger\logs\ |
| macOS    | ~/Library/Logs/VideoMerger/ |
| Linux    | ~/.config/VideoMerger/logs/ |

Renderer DevTools ( `Ctrl+Shift+I` ) for `[Preview]` , `[Auth]` , and Phase 1–4 logging tags. Python subprocess output is forwarded to the main log under each `merge-videos` IPC invocation.

---

## 7. Conclusion & Future Roadmap

---

### 7.1 Technical Debt Remaining

| Item                                 | Severity                     | Notes                                                                                                                                                       |
|--------------------------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Installer code signing               | Medium                       | Currently unsigned <code>.exe</code> ; users see SmartScreen warning. Resolution requires a code-signing certificate the team does not yet own.             |
| Caption burn-in (Phase 4b)           | Low                          | Sidecar <code>.srt</code> works today; burning into the video would require an extra FFmpeg <code>subtitles</code> filter pass. Punted to keep editability. |
| Mid-clip silence removal             | Low                          | Phase 3 trims edges only. Mid-clip removal requires synchronized <code>select / aselect</code> filter expressions. Sidestepped to preserve A/V sync.        |
| Auto-update channel                  | Medium                       | No update server; rolls forward via manual installer redistribution.                                                                                        |
| Cloud-side processing                | High (for global scale only) | Today every clip is processed on the user's machine. Multi-tenant deployment would require a queue + worker farm.                                           |
| <code>replaceAll</code> TS lib error | Trivial                      | One pre-existing renderer tsc error on <code>tsconfig.json</code> lib config. Cosmetic; does not block builds because Vite handles transpilation.           |

### 7.2 What "Version 3.0" Would Require

A globally deployed VideoMerger would need:

1. **Cloud processing tier.** The `IVideoProcessingStrategy` interface was specifically designed for this — swap `FFmpegProcessingStrategy` for an `HttpAPIStrategy` that POSTs the clip + edits payload to a worker farm. The clean-architecture core would not need changes.
2. **Object storage** for input clips and output renders. The existing `IVideoRepository` interface abstracts this; a `CloudVideoRepository` implementation drops in.
3. **Multi-tenant auth + quotas.** Google OAuth becomes one of several IDPs; a per-account quota system tracks transcoding minutes.
4. **Auto-updater.** Electron-builder supports a publish-to-GitHub or generic update server; needs a code-signing certificate first.
5. **Stress test against representative concurrent workload.** The current Phase 5b harness measures single-user wall-clock time. A V3 harness would measure queue depth × worker count × p99 latency under sustained load.
6. **Observability stack.** Today's per-user main-process logs become inadequate at scale; aggregating into Loki / Datadog / similar would replace the file-based pattern.

### 7.3 Closing Notes

The SE2 cycle delivered every panelist ask without expanding the app into a full editor: per-clip trim/crop/aspect/volume/color, EBU R128 loudness normalization, automatic silence trimming, offline auto-

captions. The codebase grew from a single Python script to a clean-architecture TypeScript core with 130 unit-test cases, an Electron build pipeline in CI, an encrypted token store, and operational documentation.

The Version 2.0 baseline is production-ready as a per-user desktop application. The same core is one adapter swap away from being the engine of a multi-tenant V3.